

Homework6 2023202300 张昕跃

Q1

Consider the following program:

```
#define LEN 10
int a[LEN][LEN];
void f(void)
{
    int i, j;
    for (i = 0; i < LEN; i++)
        for (j = 0; j < LEN; j++)
        {
            a[i][j] = i * LEN + j;
        }
}
```

Suppose the address of a is 0x10000000. After the function f() finished, fill the following table (if you don't know the value, please write NONE):

Answer1

Place	Value
%eax	0x10000000
%ecx	22
\$0x10000004	0x10000004
0x10000012	0x00
0xFFFFFFFF8	NONE
(%eax,%ecx,8)	0x2c

1. \$0x10000004是立即数，直接写立即数的值即可。

2. 对于a[i][j], 他的地址计算公式为 $a+(i*10+j)*4$, 因此0x10000012对应10进制的18。
 $18=4*4+2$
也就是第1行, 第5个数的第三个字节开始。x86是小端序, 因此从0x10000010开始应该存低地址也就是04, 所以0x10000012中存的是00。
3. 一个不知道取值的地址, 填写NONE。
4. 先算出寄存器中存的地址运算后应该是 $0x10000000+8*22$, 因此需要偏移44个数。一行有10个数, 因此应该在第五行, 第五个数, 也就是 $4*10+4=44$, 也就是0x2c

Q2

Fill the blanks of the C program:

```
int dw_loop(int x, int y, int n)
{
    do
    {
        /* */
    } while (/* */);
    return x;
}
```

The assembly code is as follows:

```
x@%ebp+8, y@%ebp+12, n@%ebp+16

movl 8(%ebp), %eax // x
movl 12(%ebp), %ecx // y
movl 16(%ebp), %edx // n
.L2:
addl %edx, %eax
imull %edx, %ecx
subl $1, %edx
testl %edx, %edx
```

```
jle .L5
cmpl %edx, %ecx
jl .L2
.L5:
```

Answer2

比较简单，直接翻译成do-while循环即可

```
int dw_loop(int x, int y, int n)
{
    do
    {
        x += n;
        y *= n;
        n--;
    } while (n > 0 && y < n);
    return x;
}
```

Q3

After ICS class, Barathrum has written a function like below:

```
int cmov_complex(int x, int y)
{
    return x < y ? x * y : (x + y) * y;
}
```

(1). Please write down the corresponding assembly code by using conditional move operations.

Answer3(1)

直接翻译，这里选用临时寄存器是%eax和%edx

```

cmov_complex:
    movl %edi, %eax // 获取参数x
    imull %esi, %eax // 计算x*y
    movl %edi, %edx
    addl %esi, %edx // 计算x+y
    imull %esi, %edx // 计算(x+y)*y
    cmpl %edi, %esi // 如果y<=x
    cmovle %edx, %eax
    ret

```

(2). When Barathrum compiles it with gcc, he finds that there's no cmov at all in the assembly code! Please explain why gcc doesn't use conditional move operations in this case.

Answer3(2)

使用条件跳转需要计算两个结果，而乘法运算的cpu时钟消耗很大（相比于判断两个数的大小关系），因此这里先判断大小关系选择分支然后沿这一分支继续运行。

Q4

Translate the following switch statements into assembly using jump table.

```

int x = <some value>;
int result = 0;
switch (x)
{
case 24:
    result = x + x;
    break;
case 27:
case 28:
    result = x + 10;
    break;
case 26:
    result = x * 2;

```

```

// Notice: there is no break here!
case 29:
case 30:
    result = result + 5;
    break;
default:
    result = 3;
    break;
}

```

Answer4

```

// Jump Table
.section .rodata
.align 4

.Ljump_table:
    .long .case_24
    .long .case_default // 25以及其他
    .long .case_26
    .long .case_27_28
    .long .case_29_30

switch_func:
    movl $0, %eax // result = 0
    movl <some value>, %ebx // x = <some value>
    subl $24, %ebx
    cmpl $6, %ebx
    ja    .case_default // 如果x不在24-30, 跳转default
    jmp *.Ljump_table(, %ebx, 4) // 跳转到 jump_table[x]

.case_24:
    // result = x + x
    leal 0x30(%ebx,%ebx), %eax // 48=32+16=0x30
    ret

```

```
.case_27_28:
    // result = x + 10;
    leal 0x22(%ebx), %eax // 10+24=34=32+2
    ret

.case_26:
    // result = x * 2
    leal 0x30(%ebx,%ebx), %eax // 48=32+16=0x30
    // 继续下一个case

.case_29_30:
    // result = result + 5;
    addl $0x5, %eax
    ret

.case_default:
    // result = 3;
    movl $0x3, %eax
    ret
```